

A Log Generator for Process Analytics

Gabriel da Silva Siqueira

Department of Computer Science, University of California at Santa Barbara, Santa Barbara, CA, USA {gdssiqueira}@ucsb.edu

Abstract. In the fields of Business Process Management and Process Analytics/Mining, analysis algorithms and techniques depend on log datasets of varying sizes and characteristics. Obtaining real datasets is a challenge – arising from accuracy, privacy, security, and secrecy concerns, and even unavailability in the case of newly installed processes. Synthetic datasets are frequently used by researchers as an effective alternative in developing/validating algorithms and techniques.

In this work we design and implement a general purpose business process simulator to streamline the generation of high-quality, custom log datasets. Additionally, we demonstrate an application of the simulator in tackling the business problem of making staffing decisions following the introduction of new processes in an organization. We conclude by discussing directions for improving usability of our proof-of-concept implementation.

Keywords: Business Process Management · Process Mining · Log Generation.

1 Introduction

With the increasing digitalization of business processes through the advance and pervasiveness of information systems, organizations have a clearer view into their daily operations. The field of Business Process Management focuses on the modeling and analysis of such processes [1]. As part of the BPM life-cycle, organizations routinely design and model new processes, which are subsequently implemented and executed. Their performance is monitored, with adjustments made when necessary, giving the life-cycle a new start.

Fitting well into the design and monitoring stages of the cycle, process mining is a separate domain which uses event log data to extract process-related information and understand how processes are actually being carried out in the organization [2]. Through process discovery, one can answer performance questions (e.g. identifying bottlenecks) or conformance questions (e.g. identifying deviations from the originally intended process flow), contrasting the idealized design with reality.

These complementary fields have a common need for diverse log data. When designing new ways of extracting process information [3], estimating the performance of newly designed processes, or automating log analysis [4], a variety of event log data-sets is needed to test hypotheses and validate assumptions.

Unfortunately, real data-sets are hard to come by. Companies are understandably unwilling to part with log data, due partly to concerns about violating user privacy, undermining company security and exposing trade secrets. Additionally, with real data-sets, flawed information systems or incomplete logging can cause the data to be unreliable. For these reasons, a first step in developing the algorithms and techniques cited involves the use of synthetic data. Besides avoiding the aforementioned concerns, synthetic data can be crafted to match specific requirements and used to test multiple scenarios during algorithm development or process design. Those data-sets can also be arbitrarily large and complex, as the researcher has complete control over its composition.

1.1 Generating Process Logs

Log generators are tools that automate the task of creating log data-sets, often by simulating the execution of multiple business processes over a period of time. Those have the potential to reduce the development time necessary to put together a comprehensive suite of log data-sets, while allowing for specific characteristics to be modelled.

PLG [5] is a project that tackles this problem. It enables the random generation of process models according to a set of input parameters, such as sequence length, number of branches, etc. The generated processes can then be simulated to create a log file. PLG only allows the simulation of one process model per log and does not model resources, such as employees. It's not suited to generate logs of complex business environments.

Commercial tools can also be used to simulate process execution. Those are typically embedded into larger suites of features and can have prohibitive cost. One example is the Aurea CX Process¹.

Lastly, a variety of research projects in the business process simulation area can be used to some extent to generate logs, but lack simulation features or offer narrow modeling capabilities. Reference [6] provides a survey of the existing projects.

In this project we design and implement an open-source² tool for general purpose process simulation and log generation. We define a model for business processes, resources, and data objects, allowing for great flexibility in the representation of complex organizational settings. We study its application to the business problem of making hiring decisions following the introduction of a new process in an organization.

This work is organized as follows: in Section 2 we review the details of our model. Then, we describe the core aspects of the simple, but extensible tool and simulation algorithm in Section 3. We study a BPM application of the log generator in Section 4, exemplifying how the log output can be used for analyzing operational efficiency. We conclude in Section 5 by exploring new directions for expansion and improvement.

¹ <https://www.aurea.com/library/cx-process/>

² <https://github.com/GabrielSiq/LogGenerator>

2 Modeling

In this generator, a set of process models needs to be specified by the user. Their execution will be simulated under the resource constraints given, resulting in log files that detail relevant execution events. In this section, we describe the modeling of the necessary input components. Those are Process Models, Activities, Gateways, Transitions, Resources and Data Objects. We also describe the structure of the log record output.

2.1 Activities

Activities are atomic tasks carried out within a business, typically by employees or machines, for certain periods of time. They may depend on and generate data objects, and require different types of resources during execution. Not all executions are successful, so multiple attempts might be necessary for an activity to be deemed completed. We define an activity as a tuple $a = (ID, Name, Duration, Data Input, Data Output, Resource Requirements, Failure Rate, Retries, Timeout, Priority Level)$ where each element is defined as follows:

- **ID:** For a given process model, the ID uniquely identifies the activity (e.g.: review).
- **Name:** Each activity ID has an associated name, which describes the type of work performed by the activity (e.g.: Review User Information).
- **Duration:** Specifies the range of time duration for that activity. It can be represented by a constant value or by a distribution and its parameters (e.g.: Normal distribution with mean of 10 minutes and std of 2 minutes).
- **Data Input:** Specifies the set of data objects required by the activity in order to execute correctly (e.g.: The address field of an application form).
- **Data Output:** Specifies the set of data objects the activity generates or modifies during its execution (e.g.: The approval status field of an application form), and how it does so.
- **Resource Requirement:** Specifies the set of resources (human or physical) required by the activity in order to execute correctly. Each requirement contains a type, a quantity, and type-specific attributes, as described in Section 2.4 (e.g.: One support specialist, from the customer experience department).
- **Failure Rate:** Represents the percentage of times a finished activity is expected have to failed (e.g.: 7%).
- **Retries:** Number of retries allowed if the activity fails (e.g.: 5 attempts).
- **Timeout:** Maximum duration allowed for an activity, including retries. If exceeded, that execution branch won't proceed. (e.g.: 2 weeks).
- **Priority Level:** Defines priority level when there is competition for resources. Allowed values are low, normal and high.

2.2 Gateways

Gateways will make decisions on the flow path taken by a process instance. Each gateway is defined by the tuple $g = (ID, Name, Type, Gates)$ where:

- **ID:** For a given process model, the ID uniquely identifies the gateway (e.g.: `checklist_completed`).
- **Name:** For Choice gateways, describes the question it evaluates (e.g.: Is the application checklist completed?). Optional for Merge and Parallel gateways.
- **Type:** Distinguishes between the three allowed types of gateways. Choice gateways only allow the execution flow to continue through one of its output gates. Parallel gateways allow the execution flow to continue through both of its output gates. Merge gateways only have one output gate, but the execution flow is only allowed to continue once both input conditions have been satisfied.
- **Gates:** Lists the available output or input gates for each gateway. For Choice gateways, gates may have an associated probability distribution. Alternatively, a data object-based decision rule must be specified.

2.3 Transitions

Transitions connect activities or gateways to other activities or gateways. Each transition is defined by the tuple $t = (Source, Destination, Delay)$ where:

- **Source:** Identifies the activity or gateway where the transition starts.
- **Destination:** Identifies the activity or gateway where the transition ends.
- **Delay:** Specifies the time delay between the start and end of the transition (e.g.: Time to move between work sites).

2.4 Resources

Resources represent available employees, materials, or machines required by an activity execution. There are two primary types of resources: Physical and Human.

Physical Resources Materials, machines and other resources of material nature. Those are represented by:

- **ID:** Across all process models, the ID uniquely identifies a resource (e.g.: `x_ray`).
- **Type:** Describes the resource identified by the ID (e.g.: X-Ray Machine).
- **Consumable:** Distinguishes between consumable resources (e.g.: materials) and non-consumable resources (e.g.: plant equipment).
- **Delay:** Determines the expected delay in replenishing a depleted consumable resource.

Human Resources Company employees, contractors and other resources of human nature. Those are represented by:

- **Organization:** Identifies which organization an employee belongs to (e.g.: UCSB).

- **Department:** Identifies which department within an organization an employee belongs to (e.g.: Computer Science).
- **Role:** Specifies the precise function of an employee within a department (e.g. Graduate Student).
- **Availability:** Specifies the period of time during which the employee is available to perform activities (e.g.: Mon-Fri, 9am-5pm).

2.5 Data Objects

Data objects can be read, created or modified by activities during the execution of a process. In this project, data objects are treated as forms, a collection of fields. Each data object is defined by the tuple $d = (ID, Name, Fields)$ where:

- **ID:** For a given process model, the ID uniquely identifies the object (e.g.:application_form).
- **Name:** Describes the content of the form (e.g.: New Account Application).
- **Fields:** Specifies the set of fields contained in that form (e.g.: Name, Address).

2.6 Process Models

Process models are the last item of our input set and build upon all of the previous definitions. They are defined by a tuple $p = (ID, Activity Set, Gateway Set, Transition Set, Arrival Rate, Deadline)$, where:

- **ID:** Uniquely identifies a process model.
- **Activity Set:** The set of activities that may be performed during the execution of a process instance.
- **Gateway Set:** The set of gateways that may be utilized during the execution of a process instance.
- **Transition Set:** The set of transitions describing how gateways and activities are connected in this process model.
- **Arrival Rate:** Specifies how often a new process instance is initiated.
- **Deadline:** A time limit for a process execution. Process instances that surpass the limit will have their execution suspended.

Figure 1 shows a graphical representation of a process model from a Customer Support department. Only selected attributes are shown, such as Name, Duration and Resource Requirement (abbreviated as S for Support) for activities, and Arrival Rate for the process model.

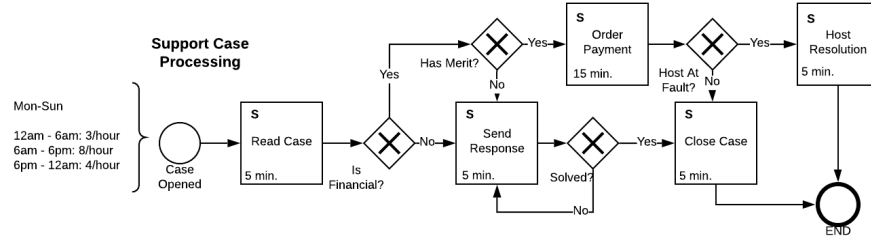


Fig. 1. A customer support process

2.7 Log Records

The log generator will output log files that reflect a simulated execution of the process models provided. During analysis, we need to extract information such as activity duration, resource and data usage, as well as differentiate between multiple process and activity instances. For that purpose, a log record is represented as a tuple $l = (LSN, Timestamp, Date, Process ID, Process Instance ID, Activity ID, Activity Instance ID, Activity Status, Data Input, Data Output, Resources)$ where each field is described below:

- **LSN**: Log Sequence Number. Global unique identifier for a log instance across the entire log file.
- **Timestamp**: UTC timestamp of when a log entry is recorded.
- **Date**: Date and time of execution of a log event.
- **Process ID, Process Instance ID, Activity ID, Activity Instance ID**: Unique identifiers for process and activity instances.
- **Activity Status**: Specifies the type of action being recorded by the log. Possible values include: Start Activity, End Activity, Pause Activity, Activity Failure, Activity Timeout, and Waiting for Resource.
- **Data Input, Data Output**: Record the values of the data objects read-/written by the activity during execution.
- **Resources**: For a given activity instance, represents the resources tied to its execution.

3 Simulator Implementation

This log generator consists of five core components that encapsulate the key functionalities described throughout this section. Those are:

- **Model Builder**: Parses all input process models described in Section 2, creating the corresponding internal representation.
- **Simulation Manager**: Controls and executes the simulation of each event, interfacing with all other modules.
- **Resource Manager**: Handles resource control and assignment.

- **Data Manager:** Creates, stores and modifies all data object instances as needed by the simulation.
- **Log Writer:** Persists event log data into structured log files.

It was implemented in Python 3.7 due to its great flexibility, active community, and abundant libraries. The project contains 25 classes across the 13 modules.

3.1 Input Parsing & Internal Representation

The first step in generating process logs is defining the simulation environment and processes involved. These are contained in 7 definition modules, which the user must supply. Those were designed to allow for high reuse, making it easier to define new processes after a set of core elements have been determined. The input files for the example from Section 4 can be found in the appendix.

- **Activities:** Global list of activities utilized by all processes.
- **Resources:** List of resource types and quantities, available in the simulation environment. These will be shared among all running process instances.
- **Data Objects:** Definitions of all form types and fields used by processes.
- **Process Models:** List of process models utilized in the simulation. References previous definitions.
- **Data Functions:** Definitions for all routines used by activities to process and modify data objects.
- **Rule Functions:** Definition for all routines used by gateways to determine execution flow based on data objects.
- **Simulation Configuration:** Additional simulation-level configuration, such as input and output path, or length of simulation.

When designing new process models, a user references the activities, resources, and data-objects defined in their respective modules. For processes that share the same or very similar activities, this design choice saves time as the set of parameters for each activity only has to be defined once. The user also has the option to overwrite specific parameters when referencing an activity, extending this benefit to activities that are partially different from the defaults.

The Model Builder parses all input files, creating internal representations that guide the simulation. For processes, activities, resources, and data objects, instances will be created to store instance-specific data, related to their state during simulation.

3.2 Control Structure

The control structure used in the Simulation Manager is central to the correct simulation of processes. In order to preserve consistency in data objects and effectively assign resources, each activity needs to be simulated with regards to their start time. Take for example activity instances A and B, where A is scheduled to be executed before B. If we were to simulate B before A, during the execution of B we might miss changes in data objects that would have happened

due to the execution of A. In the same scenario, when attempting to assign resources to B, we wouldn't know what resource instances would have been assigned to A and which would be free.

To prevent these issues, all activity instances are kept in a priority queue structure, ordered primarily by start time. The secondary ordering criteria is priority level: when multiple activities are scheduled to start at the same time but there aren't sufficient resources, the ones with higher priority will get first pick at the available resources. The third criteria is their instance IDs – this way activities that have been waiting for longer will have priority.

3.3 Activity and Gateway Simulation

Activity Simulation The simulation of an activity follows these steps:

1. Sample a duration value from the associated distribution.
2. If necessary, read required data objects/fields.
3. If necessary, assign appropriate resources.
4. Upon completion, check if timeout was reached. If so, adjust end-time and resource availability to reflect interruption upon timeout.
5. Sample a value from failure rate to determine if activity failed. If so, add the activity back to the queue, up to maximum allowed number of retries.
6. For a successfully executed activity, write all necessary data objects.
7. Follow transition originating at the current activity. Add the destination element to the execution queue.

Logs are written every time an activity starts, pauses, ends, fails, times out or has to wait for resources.

Gateway Simulation Gateways are much simpler to simulate. A gateway reads all data objects tied to the current process instance and returns the set of gates that will be activated. This set can contain one gate, in the case of Choice or Merge, or two, in the case of Parallel. The simulation takes the transition originating at each activated gate and adds the elements pointed by its destination to the execution queue.

The exception is the Merge gateway. When a Merge gateway attempts to execute, we record the input gate this execution attempt came from. If an execution request has come from all input gates (all input flows have arrived at the gate), we take the output transition and add the next element to the execution queue, much like other gateways. If not all inputs have been satisfied, we simply skip it, until another transition leads us to it again.

3.4 Resource Coordination

When attempting to assign a resource to an activity, there are three possible outcomes: a resource is available for the full duration of the activity, a resource is available for a fraction of the duration, or no resources are available.

In the first case, the resource is assigned to an activity at the start and freed at the end. In the activity's start and end logs, the resource will be indicated. We also allow for resource requirements to be partially specified. If, for example, a user specifies an organization and a department, all employees of all roles under that department will be eligible to execute the activity.

In the second case, the activity is executed for the time the resource is available. Then, it will be paused and re-inserted into the execution queue. Its new tentative start time will be the time when the next matching resource becomes available. Its new duration will be the remaining duration time. The execution logs will show a start event, followed by a pause event, with one resource instance associated to it. Additional pairs of start/pause events with different resources will follow, until the activity's full duration is met.

Lastly, if no resources are available - either for a new or previously paused activity - it will be re-inserted into the execution queue. Similarly, its new tentative start time will be the time when the next matching resource becomes available. At that time, it will have precedence over activities with the same start time, same priority level, but introduced after its first execution attempt, as explained in Section 3.2.

3.5 Data Handling

All access to data objects is handled by the Data Manager component. Each running process instance has an associated collection of data objects, which is accessed and modified during the course of the simulation. A few data access conventions are important to understanding the flow of the simulation and should be highlighted:

- Input data is read at the beginning of activity execution. Even if an activity doesn't fully execute, the data snapshot read at the beginning will be maintained for the remaining of its execution.
- Output data is written at the end of activity execution. Activities that don't fully execute will only write to data objects when they successfully complete. Failed or timed-out activities will not write data.
- Choice gateways receive all data associated with the current process instance at the time of the decision. This data might be used in rule-based Choice gateways.

3.6 Log Writing

The log writing task is straightforward. Due to the priority queue structure used to store log items during simulation, all log items are correctly ordered by the end of execution. The Log Writer iterates over the ordered log items, writing the relevant fields (Section 2.7) to a JSON file. This component can be easily extended to support additional output file formats.

3.7 Testing & Lessons

During the development of this tool, each function was tested individually using a small set of test cases. However, testing the complete tool proved a difficult task due to the large size of log outputs and the non-deterministic element introduced by the input distributions. As a way to verify some of those characteristics, we analyzed the output logs as a whole, looking to validate if the trends observed matched the specified distributions.

Another valuable ally in guaranteeing correctness were the type hints, introduced in version 3.5 of Python. Python is a dynamically typed language and, as such, names are bound to objects only at execution time. For larger projects, it becomes troublesome to identify an issue in program logic when an unexpected type is passed to a function. This difficulty is increased when the exception happens seldom, in the middle of a very large process simulation. The non-deterministic aspect mentioned before makes it harder to exactly reproduce those settings. Type hints allow the user to specify the expected type or types (multiple allowed) of variables, function return types, etc. This functionality has no effect in the program execution, but modern IDEs use type hints to perform static analysis and identify potential scenarios where a potential execution sequence might violate the expected types, alerting the user in the process. Coupled with the static analysis capabilities of the PyCharm IDE, Python type hints were instrumental in verifying code consistency.

4 Case Study: Short-Term Rental Company

Despite being originally intended as a general tool to more easily generate custom log data-sets for BPM and process mining research, due to its process simulation capabilities our tool can be used towards other business applications. Examples include identifying underutilized resources or bottlenecks, calculating business metrics under different scenarios, simulating the state of data objects involved in different processes, or estimating the impact of process changes or introductions.

We now look at the fictitious case study of a short-term rental company. The company's Support department is responsible for triaging all incoming support tickets, routing to the appropriate departments and solving the ones specific to customer experience. Those are further categorized into regular support tickets or special tickets where there is a concern about trust or safety in the platform. There are currently three processes in place in the Support department: one for the triage of incoming tickets and one for the processing of each of the two types of support tickets described. The team currently has two types of employees: the more general Support employees handle most of the activities relative to triage and regular support tickets, while specialized Trust employees address the more sensitive trust and safety cases. Figure 2 illustrates this setting.

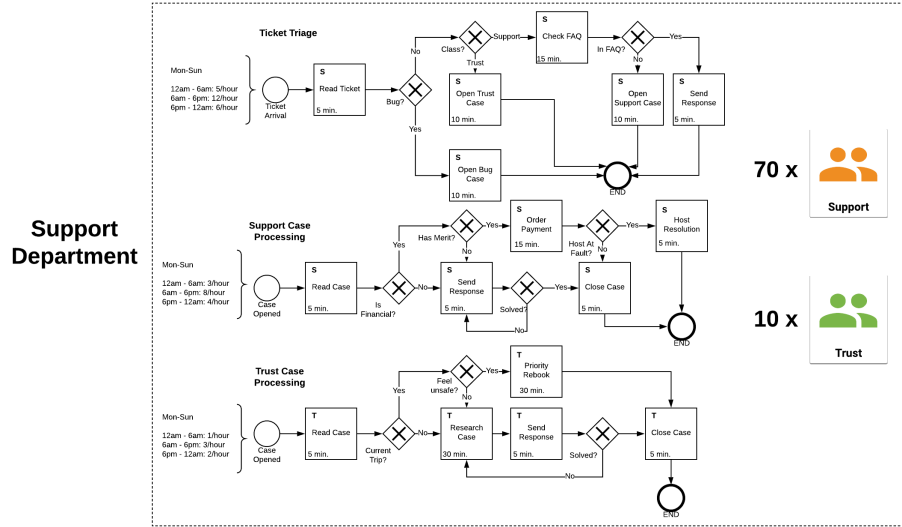


Fig. 2. Processes currently in place.

Due to the more critical nature of the issues involved, the company tracks the mean resolution time (MRT) for trust and safety cases – that is, the time between its first arrival at triage and when the case is closed. Internal guidelines specify that the MRT for these cases should always fall under 3.5 hours. With a current staff of 70 Support employees and 10 Trust employees, the team maintains an MRT of 3 hours, satisfying the established limit.

The company will start to perform background checks on new users. This process will require both Support and Trust employees and its addition to the team’s responsibilities is certain to impact the execution of current processes. Management would like to know:

1. What will be the impact of this new process introduction on the MRT for trust and safety cases?
2. How many new employees of each role will the company need to hire to keep the metric at acceptable levels?
3. How do resource utilization levels change as new employees are hired?

4.1 The Simulation Experiment

Our approach to this problem is to simulate all processes, existing and new, on the same environment, sharing the same resources, on a given time window. This will give us insight into the execution when the new process is added. Then, we can analyze the execution logs to calculate the mean resolution time for Trust cases. This will simply be the mean time between the start of the *Read Ticket* activity and end of the *Open Trust Case* activity, for the *Ticket Triage* process of Trust cases, added to the mean time between the start of the *Read Ticket* activity and end of the *Close Case* activity, for the *Trust Case Processing*

process. Our detailed execution logs record the start and end of every activity, allowing for this type of analysis. We then generate logs for other scenarios, keeping the same settings but varying the number of available Support and Trust employees. By comparing the metrics across different configurations, we can see the impact hiring new employees has on the MRT. Additionally, we can calculate the average utilization rate for Support and Trust employees for each configuration by taking the average of each individual employee's utilization. That can be obtained by dividing the number of hours during which they were assigned to an activity by the number of available work hours they have during that time-span.

Before diving into the metrics, we need to decide how long of a simulation window is required. A larger window offers the advantage of observing the long-term effects of this process introduction, with the trade-off of increased simulation run-time. To make that determination, we chose a few combinations of employee numbers and simulate for 1, 7, 14, 21 and 30 day windows. Since both our arrival rate and our employee availability information are specified as weekly pattern, we expect that a minimum of 7 days will be required for stability. In fact, Table 1 shows us that a 21-day window is enough to achieve a certain degree of stability for most configurations.

Table 1. MRT vs Simulation Window

Resources\Time	1 Day	7 Days	14 Days	21 Days	30 Days
Scenario 1	5h 46m	15h 41m	18h 45m	1d 13h 16m	1d 5h 7m
Scenario 2	7h 3m	14h 11m	13h 44m	12h 51m	12h 41m
Scenario 3	7h 27m	4h 33m	4h 42m	4h 39m	4h 52m
Scenario 4	6h 15m	3h 23m	3h 29m	3h 31m	3h 46m
Scenario 5	5h 6m	3h 10m	3h 11m	3h 12m	3h 23m

We generate logs and calculate metrics for 30 scenarios with a 30-day simulation window. For each new scenario we increase the number of Trust or Support employees by 10, alternatingly. Table 2 shows us the MRT variation as we add more employees. The complete test had a measured run-time of about 7 hours. With only the original 10 Trust employees, adding Support employees has no effect on the MRT - Trust employees are the bottleneck. At around 30 Trust employees, we start to see the metric around more familiar numbers. At that point, adding Support employees starts to generate improvement as well. We reach the desired MRT at 30-40 Trust Employees and 100-110 Support employees.

Table 2. MRT vs Resources

Trust \Support	70 (+0)	80 (+10)	90 (+20)	100 (+30)	110 (+40)
10 (+0)	1d 5h 7m	1d 10h 32m	1d 7h 2m	1d 8h 18m	1d 15h 21m
20 (+10)	12h 14m	12h 41m	12h 31m	12h 29m	12h 59m
30 (+20)	5h 19m	4h 56m	4h 52m	4h 41m	4h 36m
40 (+30)	4h 41m	4h 2m	3h 49m	3h 46m	3h 33m
50 (+40)	4h 43m	3h 57m	3h 37m	3h 25m	3h 23m
60 (+50)	4h 43m	3h 58m	3h 31m	3h 19m	3h 19m

Despite reaching desired levels of MRT, we notice that for configurations with more than 30 Trust employees and 70 Support employees, the improvements are considerably slower. This leads us to believe that we are at a point of diminishing returns, where it takes a big increase in employees to move the metric. We can also suppose that the utilization isn't very high for each resource category past that point. Table 3 show us that, after that mark, both resources are at around 60% utilization. It's an indication that the value of 3.5h for this metric must be carefully studied, as it might be expensive or wasteful to fully attain.

Table 3. Utilization vs Resources

Trust/Support	70 (+0)		90 (+20)		110 (+40)	
	Sup.	Tru.	Sup.	Tru.	Sup.	Tru.
10 (+0)	60%	96%	47%	96%	38%	96%
30 (+20)	61%	61%	48%	61%	39%	61%
50 (+40)	61%	36%	48%	37%	39%	37%

5 Conclusions & Future Work

This project presented the modeling and implementation of a general purpose tool to simulate business processes and generate their logs. It also provided a practical example of how this tool can be applied to an industry problem. Using this log generator, synthetic log data-sets for complex processes can be built quickly and reliably, allowing for the focus to remain on the research question at hand. It also avoids the prohibitive costs of existing commercial solutions, which are often full suites geared towards industry use. In many cases, synthetic data does not replace the use of real data, but serves as a helpful first step in testing algorithms and validating results.

5.1 Future Work

Usability Despite maximizing reuse by defining a master list of activities that can be referenced during the creation of new process models, specifying models still requires considerable upfront work. Several approaches found in commercial products can be studied for mitigating this issue. One possible solution consists of adding support for BPMN compliant files created by other BPM tools. When a user already has a process model created, they can simply import it into the tool and add the generator-specific parameters. An easy way to do that would be through a graphical interface. Displaying an interactive graphical representation of the process model would not only help diagnose modeling errors more easily, but it will speed up the process of specifying and modifying process models.

Scale During the case study from Section 4, we also observed that highly resource-constrained scenarios take significantly longer to run. A very constrained scenario simulation took 5 times longer to run than an adequately staffed scenario, for the same simulation window and process arrival rate. Due to the way

a lack of resources is handled, during times of low resource availability, a large number of activities will accumulate in the queue and the simulator will attempt to run each of them every time a new resource frees up. This could be mitigated by creating queues for each resource type. Only one item of the queue would be selected for execution every time a new resource is freed, significantly reducing the simulation overhead.

Events Intermediate events aren't currently supported in our model. Those are useful in complex business processes to represent external actions, such as a customer request, a phone call received, an error, an accident, etc. The incorporation of intermediate events would make for a more expressive modeling language.

Improved Resource Modeling The current prototype only accepts one type of resource per activity. We propose two types of multi-resource support for future implementation. Inclusive multi-resource requirements represent situations where more than one type of resource is required to perform an activity (e.g.: designing a web-app feature might require two engineers and one designer). Exclusive multi-resource requirements represent situations where either type of resource can satisfy an activity's requirement (i.e.: check-out can be done by a cashier or a store manager, if all cashiers are busy). Additional resource modeling guidelines have been proposed in [7] and their incorporation should also be investigated.

References

1. W. v. D. Aalst, "Business process management," 2009.
2. W. v. d. Aalst, *Process mining : data science in action*. Heidelberg: Springer, second edition. ed., 2016.
3. Y. Tang, I. Mackey, and J. Su, "Querying workflow logs," *Information*, vol. 9, no. 2, p. 25, 2018.
4. H. H. Nguyen, M. Dumas, A. H. Hofstedeter, M. L. Rosa, and F. M. Maggi, "Mining business process stages from event logs," in *29th International Conference on Advanced Information Systems Engineering (CAiSE2017)*, (Essen, Germany), 2017.
5. A. Burattin and A. Sperduti, "Plg: a process log generator," tech. rep., Citeseer, 2010.
6. M. Jansen-Vullers and M. Netjes, "Business process simulation—a tool survey," in *Workshop and Tutorial on Practical Use of Coloured Petri Nets and the CPN Tools*, Aarhus, Denmark, vol. 38, 2006.
7. W. M. P. Aalst, J. Nakatumba-Nabende, A. Rozinat, and N. Russell, "Business process simulation: How to get it right?," pp. 313–338, 01 2010.

Appendix A

In this appendix we include abbreviated copies of the input files referenced in Section 3, modeling the processes from Section 4.

activities.xml

```

1 <Activities>
2 <!-- Message Triage-->
3   <Activity id="read">
4     <Name>Read Ticket</Name>
5     <Duration>
6       <Distribution type="Normal" mean="300" std="180" />
7     </Duration>
8     <DataOutput>
9       <DataObject id="ticket" />
10    </DataOutput>
11    <Resources>
12      <Resource class_type="human" org="Company, Inc." dept="Customer
13        Experience" role="Support">1</Resource>
14    </Resources>
15  </Activity>
16  <Activity id="send_to_eng">
17    <Name>Send to Engineering</Name>
18    <Duration>
19      <Distribution type="Normal" mean="600" std="180" />
20    </Duration>
21    <Resources>
22      <Resource class_type="human" org="Company, Inc." dept="Customer
23        Experience" role="Support">1</Resource>
24    </Resources>
25  </Activity>
26  <Activity id="open_trust">
27    <Name>Open Trust Case</Name>
28    <Duration>
29      <Distribution type="Normal" mean="600" std="180" />
30    </Duration>
31    <Resources>
32      <Resource class_type="human" org="Company, Inc." dept="Customer
33        Experience" role="Support">1</Resource>
34    </Resources>
35  </Activity>
36  <Activity id="check_faq">
37    <Name>Check FAQ</Name>
38    <Duration>
39      <Distribution type="Normal" mean="900" std="180" />
40    </Duration>
41    <Resources>

```

```

39         <Resource class_type="human" org="Company, Inc." dept="Customer
           Experience" role="Support">1</Resource>
40     </Resources>
41 </Activity>
42 <Activity id="open_support">
43     <Name>Open Support Case</Name>
44     <Duration>
45         <Distribution type="Normal" mean="600" std="180" />
46     </Duration>
47     <Resources>
48         <Resource class_type="human" org="Company, Inc." dept="Customer
           Experience" role="Support">1</Resource>
49     </Resources>
50 </Activity>
51 <Activity id="send_response">
52     <Name>Send Response</Name>
53     <Duration>
54         <Distribution type="Normal" mean="300" std="180" />
55     </Duration>
56     <Resources>
57         <Resource class_type="human" org="Company, Inc." dept="Customer
           Experience" role="Support">1</Resource>
58     </Resources>
59 </Activity>
60 </Activities>

```

resources.xml

```

1 <Resources>
2     <Resource type='human' id='support '>
3         <Organization>Company, Inc.</Organization>
4         <Department>Customer Experience</Department>
5         <Role>Support</Role>
6         <Quantity>70</Quantity>
7         <Availability>
8             <Mon>
9                 <TimeBlock start="0" end="2" />
10                <TimeBlock start="9" end="12" />
11                <TimeBlock start="13" end="17" />
12                <TimeBlock start="18" end="22" />
13                <TimeBlock start="23" end="24" />
14            </Mon>
15            <Tue>
16                <TimeBlock start="0" end="2" />
17                <TimeBlock start="9" end="12" />
18                <TimeBlock start="13" end="17" />
19                <TimeBlock start="18" end="22" />
20                <TimeBlock start="23" end="24" />

```



```

21     </Tue>
22     <Wed>
23         <TimeBlock start="0" end="2" />
24         <TimeBlock start="9" end="12" />
25         <TimeBlock start="13" end="17" />
26         <TimeBlock start="18" end="22" />
27         <TimeBlock start="23" end="24" />
28     </Wed>
29     <Thu>
30         <TimeBlock start="0" end="2" />
31         <TimeBlock start="9" end="12" />
32         <TimeBlock start="13" end="17" />
33         <TimeBlock start="18" end="22" />
34         <TimeBlock start="23" end="24" />
35     </Thu>
36     <Fri>
37         <TimeBlock start="0" end="2" />
38         <TimeBlock start="9" end="12" />
39         <TimeBlock start="13" end="17" />
40         <TimeBlock start="18" end="22" />
41         <TimeBlock start="23" end="24" />
42     </Fri>
43     <Sat>
44         <TimeBlock start="0" end="2" />
45         <TimeBlock start="10" end="12" />
46         <TimeBlock start="13" end="15" />
47         <TimeBlock start="18" end="22" />
48         <TimeBlock start="23" end="24" />
49     </Sat>
50     <Sun>
51         <TimeBlock start="0" end="2" />
52         <TimeBlock start="10" end="12" />
53         <TimeBlock start="13" end="15" />
54         <TimeBlock start="18" end="22" />
55         <TimeBlock start="23" end="24" />
56     </Sun>
57 </Availability>
58 </Resource>
59 </Resources>

```

data.xml

```

1 <DataObjects>
2   <DataObject id="ticket" type="form">
3     <Name>Ticket</Name>
4     <Fields>
5       <Field name="Class"></Field>
6     </Fields>

```

```

7     </DataObject>
8 </DataObjects>

```

models.xml

```

1 <Models>
2   <Model id='trriage '>
3     <Name>Ticket Triage</Name>
4     <ArrivalRate>
5       <Mon>
6         <TimeBlock start="0" end="6">5</TimeBlock>
7         <TimeBlock start="6" end="18">12</TimeBlock>
8         <TimeBlock start="18" end="24">6</TimeBlock>
9       </Mon>
10      <Tue>
11        <TimeBlock start="0" end="6">5</TimeBlock>
12        <TimeBlock start="6" end="18">12</TimeBlock>
13        <TimeBlock start="18" end="24">6</TimeBlock>
14      </Tue>
15      <Wed>
16        <TimeBlock start="0" end="6">5</TimeBlock>
17        <TimeBlock start="6" end="18">12</TimeBlock>
18        <TimeBlock start="18" end="24">6</TimeBlock>
19      </Wed>
20      <Thu>
21        <TimeBlock start="0" end="6">5</TimeBlock>
22        <TimeBlock start="6" end="18">12</TimeBlock>
23        <TimeBlock start="18" end="24">6</TimeBlock>
24      </Thu>
25      <Fri>
26        <TimeBlock start="0" end="6">5</TimeBlock>
27        <TimeBlock start="6" end="18">12</TimeBlock>
28        <TimeBlock start="18" end="24">6</TimeBlock>
29      </Fri>
30      <Sat>
31        <TimeBlock start="0" end="6">5</TimeBlock>
32        <TimeBlock start="6" end="18">12</TimeBlock>
33        <TimeBlock start="18" end="24">6</TimeBlock>
34      </Sat>
35      <Sun>
36        <TimeBlock start="0" end="6">5</TimeBlock>
37        <TimeBlock start="6" end="18">12</TimeBlock>
38        <TimeBlock start="18" end="24">6</TimeBlock>
39      </Sun>
40    </ArrivalRate>
41    <Activities>
42      <Activity id="read">
43        <DataOutput>

```

```

44         <DataObject id="ticket">
45             <Fields>
46                 <Field name="Class" />
47             </Fields>
48         </DataObject>
49     </DataOutput>
50 </Activity>
51 <Activity id="send_to_eng" />
52 <Activity id="open_trust" />
53 <Activity id="check_faq" />
54 <Activity id="open_support" />
55 <Activity id="send_response" />
56 </Activities>
57 <Gateways>
58     <Gateway id="bug">
59         <Name>Is it a bug?</Name>
60         <Type>Choice</Type>
61         <Rule>
62             <Gate id="yes" />
63             <Gate id="no" />
64         </Rule>
65     </Gateway>
66     <Gateway id="type">
67         <Name>What is the type?</Name>
68         <Type>Choice</Type>
69         <Rule>
70             <Gate id="support" />
71             <Gate id="trust" />
72         </Rule>
73     </Gateway>
74     <Gateway id="faq">
75         <Name>Is it in FAQ?</Name>
76         <Type>Choice</Type>
77         <Distribution>
78             <Gate id='yes'>0.4</Gate>
79             <Gate id='no'>0.6</Gate>
80         </Distribution>
81     </Gateway>
82 </Gateways>
83 <Transitions>
84     <Transition source="START" destination="read">
85         <Duration>
86             <Distribution type="Const" value="10" />
87         </Duration>
88     </Transition>
89     <Transition source="read" destination="bug">
90         <Duration>
91             <Distribution type="Const" value="10" />
92         </Duration>
93     </Transition>

```

```

94     <Transition source="bug" source_gate="yes" destination="
95         send_to_eng">
96         <Duration>
97             <Distribution type="Const" value="10" />
98         </Duration>
99     </Transition>
100    <Transition source="bug" source_gate="no" destination="type">
101        <Duration>
102            <Distribution type="Const" value="10" />
103        </Duration>
104    </Transition>
105    <Transition source="type" source_gate="support" destination="
106        check_faq">
107        <Duration>
108            <Distribution type="Const" value="10" />
109        </Duration>
110    </Transition>
111    <Transition source="type" source_gate="trust" destination="
112        open_trust">
113        <Duration>
114            <Distribution type="Const" value="10" />
115        </Duration>
116    </Transition>
117    <Transition source="send_to_eng" destination="END">
118        <Duration>
119            <Distribution type="Const" value="10" />
120        </Duration>
121    </Transition>
122    <Transition source="open_trust" destination="END">
123        <Duration>
124            <Distribution type="Const" value="10" />
125        </Duration>
126    </Transition>
127    <Transition source="check_faq" destination="faq">
128        <Duration>
129            <Distribution type="Const" value="10" />
130        </Duration>
131    </Transition>
132    <Transition source="faq" source_gate="yes" destination="
133        send_response">
134        <Duration>
135            <Distribution type="Const" value="10" />
136        </Duration>
137    </Transition>
138    <Transition source="faq" source_gate="no" destination="
139        open_support">
140        <Duration>
141            <Distribution type="Const" value="10" />
142        </Duration>
143    </Transition>

```

```
139     <Transition source="send_response" destination="END">
140         <Duration>
141             <Distribution type="Const" value="10" />
142         </Duration>
143     </Transition>
144     <Transition source="open_support" destination="END">
145         <Duration>
146             <Distribution type="Const" value="10" />
147         </Duration>
148     </Transition>
149 </Transitions>
150 <Deadline>500000000</Deadline>
151 </Model>
152 </Models>
```